Document 07

Output Specification

1. Purpose of This Document

This document defines the exact machine readable output contract for Trinity Engine v1.

Its purpose is to specify:

- the canonical output structure
- required top level sections
- required field names
- identifier formats
- ordering rules
- deterministic metadata fields
- error response structure
- domain specific output payloads for Astrology, Human Design, and Gene Keys

This document defines what the engine must return and how that output must be structured.

It does not define presentation formatting, content rendering, translations, or human facing explanation.

The canonical output defined in this document is intended to serve as stable machine readable downstream input for non canonical product layers such as thematic mapping, survey enrichment, and content assembly, without changing the authoritative calculation boundary of Trinity Engine v1.

---

2. General Output Principles

2.1 Machine Readable Output Rule

The engine must return machine readable structured output only.

The engine must not return:

- prose
- interpretation text
- translated labels
- HTML
- Markdown
- UI formatting

- coaching language
- advice
- presentation layer aliases

2.2 Canonical Field Rule

Only canonical field names defined in this document are authoritative for Trinity Engine v1.

No implementer may:

- rename fields for convenience
- emit multiple aliases for the same field
- change casing conventions
- omit required sections
- invent additional output sections that appear canonical

2.3 Stable Ordering Rule

Where output contains ordered collections, those collections must use the canonical order defined in this document.

No implementer may:

- sort by convenience
- sort alphabetically unless canon explicitly requires it
- emit non deterministic order based on hash maps, object iteration, or library behavior

2.4 Canonical Case Rule

For v1, all named field identifiers and object identifiers must use:

- lowercase snake_case for field names and named identifiers
- integers for gate and line values where applicable
- ISO style date and time strings where specified
- hyphen joined ascending gate pairs for channel identifiers

---

3. Top Level Output Structure

3.1 Success Response Shape

A successful Trinity Engine v1 response must contain the following top level fields:

- `status`
- `metadata`
- `input_echo`
- `astrology`

- `human_design`
- `gene_keys`

3.2 Success Status Value

For successful responses:

- `status = "success"`

No alternate success label is allowed.

3.3 Canonical Success Shape Example

```
{
 "status": "success",
 "metadata": {},
 "input_echo": {},
 "astrology": {},
 "human_design": {},
 "gene_keys": {}
}
```

This example illustrates structure only.
All required subfields defined below must be present.

---

4. Metadata Output Contract

4.1 Required Metadata Fields

The `metadata` object must contain at minimum:

- `engine_version`
- `canon_version`
- `source_stack_version`
- `input_schema_version`
- `mapping_version`

These fields are mandatory for every successful response.

4.2 Metadata Field Types

All required metadata fields must be strings.

4.3 Metadata Scope Rule

Metadata must contain deterministic version references only.

The canonical output for v1 must not include:

- random run ids
- environment names
- infrastructure specific identifiers
- operational log ids
- timestamp fields unless explicitly added by later canon revision
- mutable debugging fields

4.4 Metadata Example

```
{
 "engine_version": "v1.0.0",
 "canon_version": "v1.0",
 "source_stack_version": "v1.0",
 "input_schema_version": "v1.0",
 "mapping_version": "v1.0"
}
```

---

5. Input Echo Contract

5.1 Purpose

The `input_echo` object must return the canonical engine input values actually used for calculation.

This exists to support:

- reproducibility
- debugging
- acceptance testing
- verification that upstream normalization reached the engine correctly

5.2 Required Input Echo Fields

The `input_echo` object must contain:

- `birth_date`
- `birth_time`
- `timezone`
- `latitude`
- `longitude`

These fields must reflect the normalized input accepted by the engine.

5.3 Input Echo Rules

The input echo:

- must use the same canonical formats as Document 04
- must not include extra non canonical upstream fields
- must not mutate canonical values after validation
- must not include raw birthplace text or any excluded intake fields

5.4 Timezone Identifier Echo and Acceptance Rule

The `timezone` value returned in `input_echo` must be the canonical timezone identifier accepted and used by the engine for calculation.

For Trinity Engine v1:

- only canonical IANA `Area/Location` timezone identifiers are accepted
- abbreviations are not accepted
- aliases and link names are not accepted unless explicitly added by later canon revision
- the echoed timezone value must exactly match the canonical accepted identifier used during calculation

Examples of rejected forms in v1 include:

- `CET`
- `EST`
- `US/Eastern`

---

6. Astrology Output Contract

6.1 Required Astrology Sections

The `astrology` object must contain:

- `system`
- `angles`
- `house_cusps`
- `objects`

6.2 Astrology System Section

The `system` object must contain:

- `zodiac`
- `house_system`
- `node_type`

Canonical values for v1:

- `zodiac = "tropical"`
- `house_system = "placidus"`
- `node_type = "mean"`

6.3 Astrology Angles Section

The `angles` object must contain:

- `ascendant`
- `midheaven`

Each angle object must contain:

- `longitude`
- `sign`

6.4 Astrology House Cusps Section

The `house_cusps` field must be an ordered array of 12 objects.

Each cusp object must contain:

- `house`
- `longitude`
- `sign`

Ordering rule:

- house 1 through house 12 in ascending numeric order

6.5 Astrology Objects Section

The `objects` field must be an ordered array of astrology object entries.

Each entry must contain:

- `object_id`
- `longitude`
- `sign`
- `house`

6.6 Astrology Object Order

The `objects` array must appear in this exact order:

1. `sun`
2. `moon`
3. `mercury`
4. `venus`
5. `mars`
6. `jupiter`
7. `saturn`
8. `uranus`
9. `neptune`
10. `pluto`
11. `chiron`
12. `north_node_mean`
13. `earth`

6.7 Astrology Numeric Field Rule

For astrology output:

- `longitude` must be numeric
- `house` must be integer
- `sign` must be canonical lowercase snake_case sign identifier

6.8 Astrology Sign Identifier Set

The allowed sign identifiers are:

- `aries`
- `taurus`
- `gemini`
- `cancer`
- `leo`
- `virgo`
- `libra`
- `scorpio`
- `sagittarius`

- `capricorn`
- `aquarius`
- `pisces`

---

7. Human Design Output Contract

7.1 Required Human Design Sections

The `human_design` object must contain:

- `system`
- `personality_activations`
- `design_activations`
- `channels`
- `centers`
- `definition`
- `type`
- `authority`
- `profile`
- `incarnation_cross`

7.2 Human Design System Section

7.2 Human Design System Section

The `system` object must contain:

- `node_type`
- `design_time_utc`

Canonical values:

- `node_type = "true"`
- `design_time_utc` must be the canonical UTC datetime string used for the Design snapshot

The authoritative determination of the Design time instant, including whole second emission behavior when the final Design time search interval is narrower than one second, is defined in Document 05.

No alternate rounding, midpoint selection, ceiling rule, or serializer side adjustment may alter the canonical `design_time_utc` value emitted here.

7.3 Personality Activations Section

The `personality_activations` field must be an ordered array of 13 activation objects.

Each activation object must contain:

- `object_id`
- `gate`
- `line`

The array must appear in this exact order:

1. `sun`
2. `earth`
3. `north_node`
4. `south_node`
5. `moon`
6. `mercury`
7. `venus`
8. `mars`
9. `jupiter`
10. `saturn`
11. `uranus`
12. `neptune`
13. `pluto`

7.4 Design Activations Section

The `design_activations` field must be an ordered array of 13 activation objects.

Each activation object must contain:

- `object_id`
- `gate`
- `line`

The order must be identical to the Personality activation order.

7.5 Channel Output Section

The `channels` field must be an ordered array of active channel objects.

Each channel object must contain:

- `channel_id`
- `gate_a`

- `gate_b`
- `center_a`
- `center_b`

Ordering rule:
Active channels must be returned in ascending lexicographic order by `channel_id`.

No alternate ordering is authoritative for v1.

7.6 Center Output Section

The `centers` field must be an ordered array of 9 center state objects.

Each center object must contain:

- `center_id`
- `state`

Allowed state values:

- `defined`
- `undefined`

The `centers` array must appear in this exact order:

1. `head`
2. `ajna`
3. `throat`
4. `g`
5. `ego`
6. `solar_plexus`
7. `sacral`
8. `spleen`
9. `root`

7.7 Definition Field

The `definition` field must be a string with one of the following allowed values:

- `none`
- `single`
- `split`
- `triple_split`
- `quadruple_split`

7.8 Type Field

The `type` field must be one of:

- `reflector`
- `generator`
- `manifesting_generator`
- `manifestor`
- `projector`

7.9 Authority Field

The `authority` field must be one of:

- `emotional`
- `sacral`
- `splenic`
- `ego_manifested`
- `ego_projected`
- `self_projected`
- `mental`
- `lunar`

7.10 Profile Field

The `profile` field must be a string in the format:

- `X/Y`

Where:

- `X` = Personality Sun line
- `Y` = Design Sun line

Example:

- `3/5`

7.11 Incarnation Cross Field

The `incarnation_cross` object must contain:

- `personality_sun`
- `personality_earth`
- `design_sun`

- `design_earth`

Each of those four subobjects must contain:

- `gate`
- `line`

No human readable cross name is required in v1.

---

8. Gene Keys Output Contract

8.1 Required Gene Keys Sections

The `gene_keys` object must contain:

- `system`
- `activations`

8.2 Gene Keys System Section

The `system` object must contain:

- `derivation_basis`

Canonical value:

- `derivation_basis = "human_design"`

8.3 Gene Keys Activations Section

The `activations` object must contain:

- `life_work`
- `evolution`
- `radiance`
- `purpose`

Each activation object must contain:

- `key`
- `line`

8.4 Gene Keys Mapping Rule

The values must map exactly as follows:

- `life_work` = Personality Sun
- `evolution` = Personality Earth
- `radiance` = Design Sun
- `purpose` = Design Earth

8.5 Gene Keys Identifier Rule

For v1:

- `key` must equal the corresponding Human Design `gate`
- `line` must equal the corresponding Human Design `line`

No additional Gene Keys section is in scope.

---

9. Error Output Contract

9.1 Error Response Shape

An error response must contain the following top level fields:

- `status`
- `metadata`
- `error`

9.2 Error Status Value

For error responses:

- `status = "error"`

9.3 Error Object Fields

9.3 Error Object Fields

The error object must contain:

- `error_type`
- `error_code`
- `message`

Field rules:

- `error_type` is mandatory and canonical
- `error_code` is mandatory and canonical

- `message` is mandatory but non canonical developer facing text

The `message` field exists to support developer interpretation of the error condition, but it must not be used as the authoritative basis for canonical fixture matching unless a later canon revision explicitly pins a message catalog.

9.4 Allowed Error Types

The allowed `error_type` values are:

- `invalid_input`
- `incomplete_input`
- `unsupported_input`
- `canon_conflict`
- `execution_failure`

These categories must be applied according to the canonical classification rules below.

Classification rules

- `invalid_input`
  Use when a required input field is present but malformed, structurally invalid, or outside the accepted canonical format.
- `incomplete_input`
  Use when a required canonical input field is absent or not fully provided.
- `unsupported_input`
  Use when the input is structurally valid but outside the accepted v1 support boundary.
- `canon_conflict`
  Use when canon required version references, pinned dependencies, or mutually required canon conditions are inconsistent.
- `execution_failure`
  Use only when canon compliant input and canon compliant configuration were present, but the engine still failed to complete execution for a non canonical reason.

Canonical classification examples

Scenario	Canonical error_type	Canonical error_code
`birth_date` missing	`incomplete_input`	`missing_birth_date`

`birth_time` missing	`incomplete_input`	`missing_birth_time`
`birth_time` present as `17` with missing minutes	`incomplete_input`	`incomplete_birth_time`
`birth_date` malformed	`invalid_input`	`invalid_birth_date_format`
`latitude` non numeric	`invalid_input`	`invalid_latitude_format`
timezone given as abbreviation such as `CET`	`unsupported_input`	`unsupported_timezone_identifier`
timezone given as alias such as `US/Eastern` if aliases are not accepted in v1	`unsupported_input`	`unsupported_timezone_identifier`

9.5 Error Message Rule

9.5 Error Message Rule

The `message` field must be a machine readable developer facing string.

It must not contain:

- coaching language
- user facing UI prose
- remediation advice outside engine scope
- stack traces unless explicitly enabled in non canonical operational tooling

The `message` field is not authoritative for canonical comparison. Canonical error comparison must use:

- `error_type`
- `error_code`
- any other explicitly canonical fields defined by the error contract

Free text wording in `message` may vary across implementations or revisions unless a later canon revision explicitly pins an authoritative message catalog.

9.5A Canonical Error Code Rule

Each supported canonical error condition must map to a stable `error_code`.

The `error_code` field must:

- use lowercase snake_case
- identify the canonical failure condition more specifically than `error_type`
- remain stable across implementations for the same canonical error condition

Examples:

- `invalid_latitude_format`
- `missing_birth_time`
- `unsupported_timezone_identifier`
- `canon_version_conflict`

The full error code set may be expanded by later canon revision, but any emitted `error_code` in v1 must be deterministic and canon compliant.

9.6 Error Metadata Rule

Error responses must still contain the deterministic metadata fields required by Section 4, where available.

9.7 Canonical Error Shape Example


```
{
 "status": "error",
 "metadata": {
   "engine_version": "v1.0.0",
   "canon_version": "v1.0",
   "source_stack_version": "v1.0",
   "input_schema_version": "v1.0",
   "mapping_version": "v1.0"
 },
 "error": {
   "error_type": "invalid_input",
```

```
  "message": "latitude must be numeric"
 }
}
```

---

10. Numeric Precision and Formatting Rules

10.1 Longitude Precision Rule

All output longitude values must be emitted as numeric values rounded to **6 decimal places**.

This rule applies to:

- astrology object longitudes
- angle longitudes
- house cusp longitudes
- any other canonical longitude output field

No alternate output precision is allowed in canonical engine output for v1.

10.2 Longitude Formatting Rule

Longitude values must be emitted as numeric JSON values, not strings.

Trailing zeros may appear according to implementation serialization, but the canonical numeric precision basis is 6 decimal places.

10.3 Datetime Format Rule

`design_time_utc` must be emitted in ISO 8601 UTC format with trailing `Z`.

Canonical example:

- `1990-04-09T17:04:00Z`

No alternate datetime string format is authoritative for v1.

10.4 Datetime Precision Rule

For Trinity Engine v1, `design_time_utc` must be emitted to whole second precision.

Canonical example:

- `1990-04-09T17:04:00Z`

Millisecond precision is out of scope unless added by later canon revision.

The authoritative determination of which whole second must be emitted is defined in Document 05. This document governs only output format and precision, not alternate timestamp selection logic.

10.5 Success Response Comparison Basis

Canonical success response comparison for Trinity Engine v1 must be based on semantic JSON equality against the output contract defined in this document and the governing calculation rules defined elsewhere in the canon.

This means:

- required top level fields must be present
- required subfields must be present
- canonical field names must match exactly
- exact scalar values must match after canonical numeric and datetime formatting rules are applied
- ordered arrays must preserve canonical order where this document specifies order
- object member ordering is not authoritative unless explicitly pinned by later canon revision

Literal byte for byte JSON serialization is not required for canonical success comparison unless a later canon revision explicitly pins serializer behavior and object member ordering.

---

11. Out of Scope Output Fields

The engine must not emit:

- content layer text
- sign descriptions
- gate descriptions
- Gene Keys shadow, gift, or essence content
- profile descriptions
- strategy text
- translation labels
- UI aliases
- display formatting fields
- debug only operational fields
- hashes unless added by later canon revision

---

12. Dependency Notes

12. Dependency Notes

This document must remain aligned with:

- Document 01. Purpose and System Boundary
- Document 04. Input Specification
- Document 05. Calculation Canon
- Document 06. Mapping Canon
- Document 08. Determinism and Versioning Rules
- Document 10. Golden Test Package and Acceptance Rules

This document cannot be treated as final until:

- final consistency pass is completed against Documents 05 and 06
- error comparison rules are aligned with Document 10
- success response comparison semantics are aligned with Document 10